Introduction

Tuesday, November 25, 2025 10:29 PM

Version Control System

Version control is a system that **records changes** to a file or set of files over time so that you can **recall specific versions later**. It can record changes of file of **any type** such as **source code**, **images**, **docs** etc

Advantages

1. Backup and Recovery:
    a. It allows you to **revert selected** files **back** to a **previous state**
    b. Revert the **entire project** back to a **previous state**

2. Collaboration
    a. Allows everyone to work on their own copies and **later merge changes seamlessly** to avoid conflicts

3. Tracking Changes overtime
    a. **Compare** changes over time
    b. Find out who **last modified something** that might be causing a problem, who introduced an issue and when, and more.

4. Testing and Experimentation
    a. Create **different branches or versions of project** to test new features

Question: Up till now, how you have been keeping record of changes in your file or project?

Local Version Control System

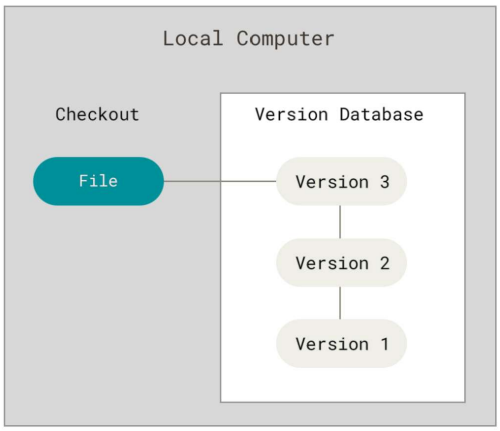
Examples: Revision Control System (RCS) etc.

1. One common & simple choice is to **copy files** into **another directory** .
    a. May include **versions** in directory name
        i. V1.0.0
        ii. V1.0.1

b. Perhaps a time-stamped directory, if you are clever
    i. 2022-09-20 19:20:35
    ii. 2022-09-19 19:20:35

2. It is incredibly **error prone**, **easy to forget** which directory you're in and **accidentally write** to the **wrong file** or copy over files you don't mean to.

Solution (Local VCs)

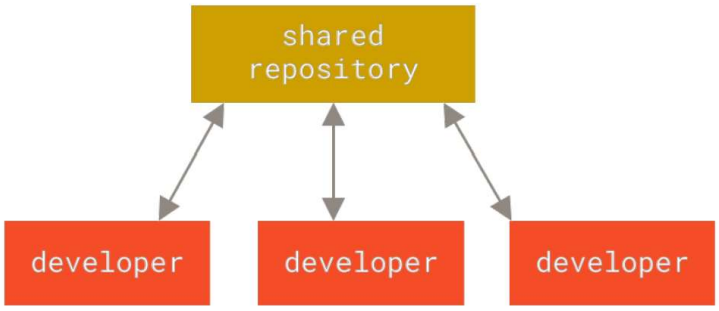


Problem

What if you need to collaborate with developers on other systems?

Centralized Version Control Systems

Examples: CVs, Subversion, Perforce etc.



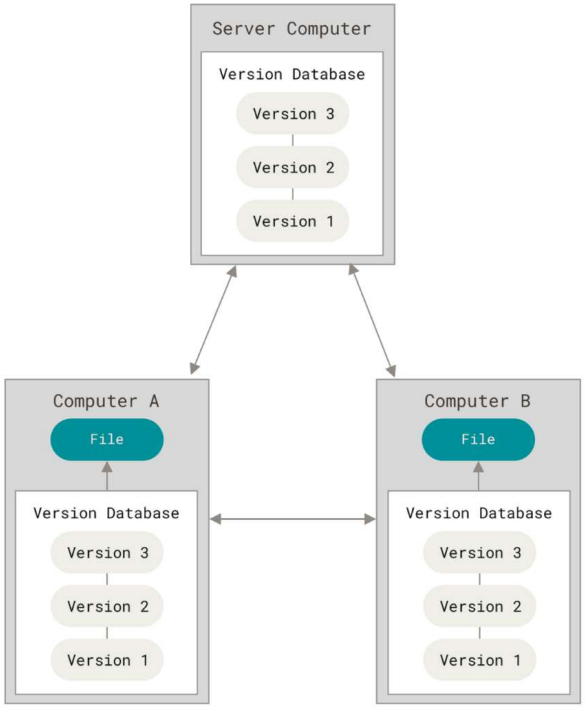
Question: Do you see any problem with this kind of structure?

a. What if server goes down for some time?
b. What if the proper server backups are not being kept and somehow data is corrupted?

Whenever you have the **entire history** of the project in a **single place**, you **risk losing everything**.

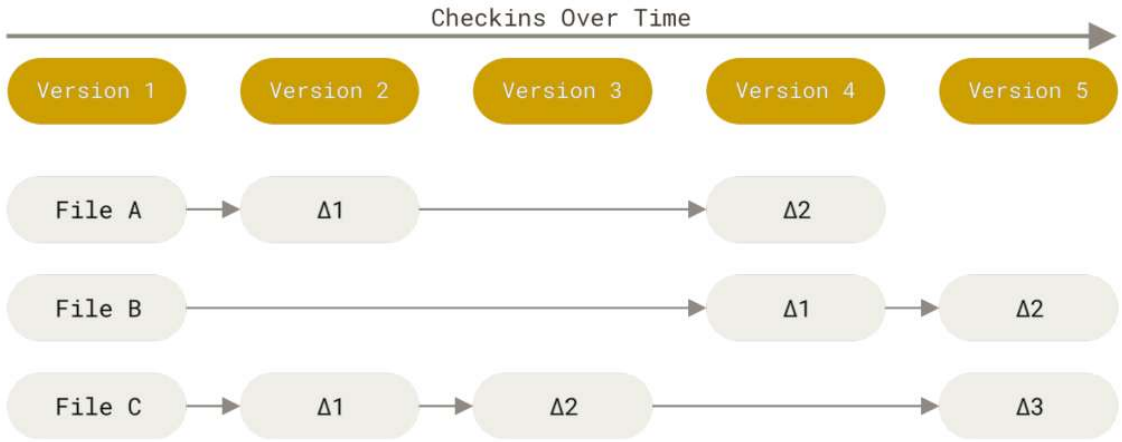
Distributed Version Control Systems

Examples: Git, Mercurial, Bazaar, Darcs, BitKeeper etc.

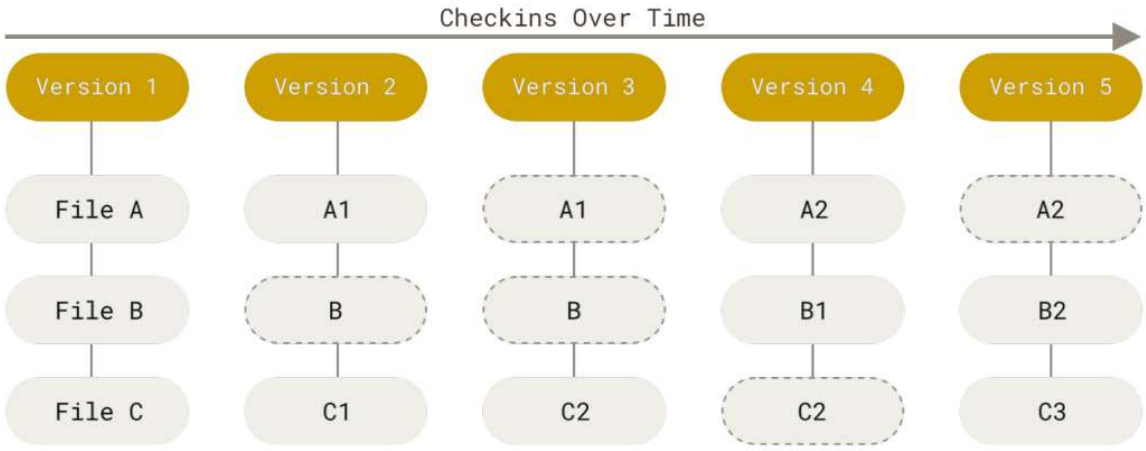


Git Records Snapshots, Not Differences

Other Version Control Systems (VCs): Store data as changes to a base version of each file



Git Version Control System: Store data as snapshots of the project over time



Nearly Every Operation is Local

Remember: Git is a **distributed version control system** which means you have the entire project history on your local machine.

1. You can **browse previous history** of the project without requesting server .
2. You may l**ook into the changes** introduced **between different versions** instantly.
3. And many other operations etc.

Almost, there is **nothing** you **can't do** if you are **offline**.

Git Has Integrity

Remember: everything in Git is **checksummed** before it is stored and is then referred to by that checksum.

1. It is **impossible** to **change** the contents of any **file** in a project without **knowing** Git.
2. Git uses **SHA-1** hash for checksum.
3. String of characters of 40 length.
    a. e.g. *24b9da6552252987aa493b52f8696cd6d3b00373*
4. Git uses this hash to store contents in a database

Sections & States of Git

<u>Remember</u>: There are **three** main **states** of git in which your particular file could be.

1. <u>Modified</u>

   You have **changed** the file but have **not committed** it to your database yet.

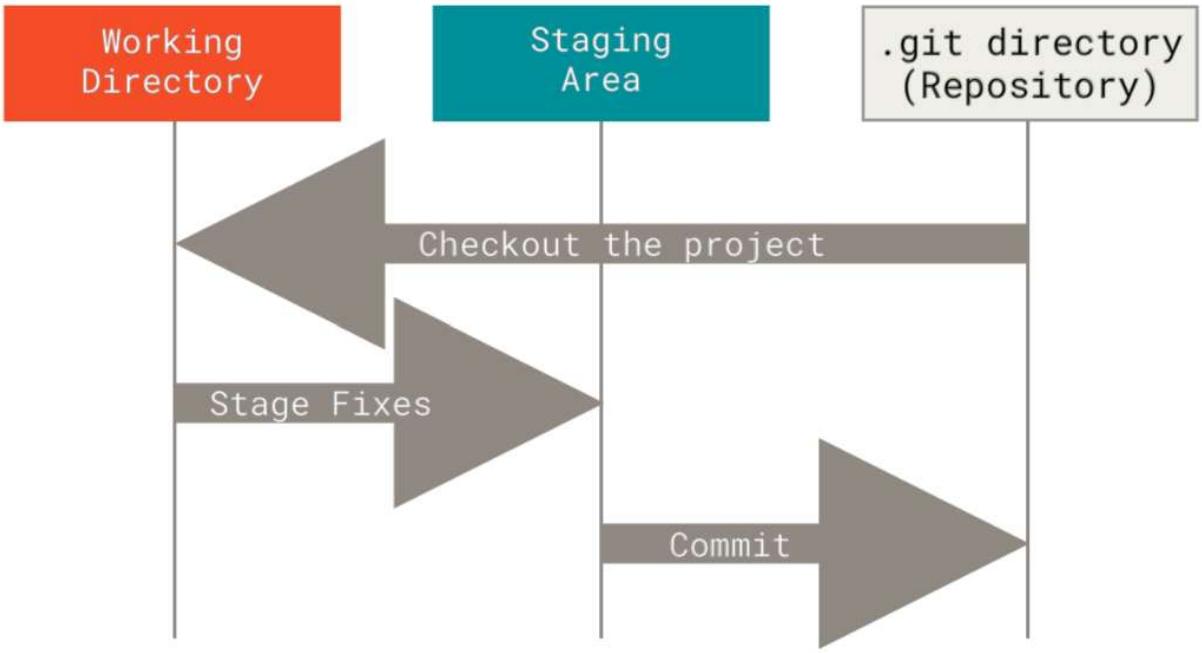
2. <u>Staged</u>

   You have **marked** a **modified** file in its **current version** to go into your **next commit** snapshot

3. <u>Committed</u>

   The data is **safely stored** in your **local database**

This leads towards **three** main **sections** of git which is **Working tree**, **staging area**, and **Git directory**



1. <u>Working Tree</u>

   is a **single checkout** of one version of the project. Files are **pulled out** from the **git directory** and placed on disk for you to **use** and **modify**

2. <u>Staging Area</u>

   is a file, generally **contained** in your **Git directory**, that **stores** information about what will go into your **next commit**.

3. <u>Git Directory</u>

is where **Git stores** the **metadata** and **object database** for your project. This is the **most important** part of Git, and it is **what is copied** when you **clone** a **repository** from another computer.

Basic Git Workflow

1. You **modify files** in your **working tree**.

2. You **selectively stage** just those **changes** you want to be part of your **next commit**, which **adds** only those changes to the **staging area**.

3. You do a **commit**, which **takes** the **files** as they are in the staging area and **stores** that **snapshot permanently** to your **Git directory**.